

MRT PROGRAMLAMA DİLİ

Resmi Başvuru ve Kullanım El Kitabı

Sürüm 0.2.0 • Saf C ile Sıfırdan Geliştirilen Bağımsız Programlama Dili ve Yerel Linux IDE'si

Dil Sürümü:	MRT 0.2.0
IDE Sürümü:	MRT Studio 0.2.0
Mimari:	Saf Standart C (C11/C17), AST Tree-Walking Yorumlayıcı
Grafik Arayüz:	GTK4 + GtkSourceView 5 (Hafif, Yerel Linux)
Geliştirici:	Murat Özçelik
Kaynak Kodu:	https://github.com/MOzcelik14/MRT
Lisans:	MIT Açık Kaynak Lisansı
Tarih:	Ekim 2026

"Bu proje basit bir parser demosu değil; temiz mimarili, genişletilebilir ve sıfır bağımlılıklı gerçek bir programlama dili implementasyonudur."

İçindekiler Tablosu

1. Giriş ve Dilin Temel Felsefesi

MRT'nin çıkış noktası, sıfır harici bağımlılık ilkesi ve temiz C mimarisi.

2. Kurulum ve Hızlı Başlangıç

Derleme, test suit, sisteme kurulum, REPL ve komut satırı argümanları.

3. MRT 0.2.0 Sözdizimi ve Veri Tipleri

Değişkenler (var), temel veri tipleri (yes, no, none), dizgeler ve blok yorumlar.

4. Operatörler ve İfadeler

Aritmetik, mantıksal, karşılaştırma operatörleri ve öncelik sıralaması.

5. Konsol Çıktısı: 'say' Deyimi

say deyiminin doğası, otomatik satır sonu ve birinci sınıf dil yapısı.

6. Kontrol Akışı (when, otherwise, repeat)

when/otherwise blokları, repeat döngüsü, break ve continue akış kontrolü.

7. Görevler (Tasks / Fonksiyonlar)

task tanımları, give deyimi, sözcüksel kapsam (lexical scope) ve özyineleme.

8. Yerleşik Standart Kütüphane

typeof, length, toText ve clock fonksiyonlarının kullanımı ve dönüş tipleri.

9. Hata Yönetimi ve Tanılama

SyntaxError, NameError, TypeError, RuntimeError ve imleçli görsel hata işaretçileri.

10. MRT Studio IDE Kılavuzu

GTK4 yerel arayüzü, çift dilli destek, komut paleti, sembol ağacı ve kurtarma.

11. Dil Mimarisi ve Gelecek Yol Haritası

Lexer -> Parser -> AST -> Interpreter ardışık düzeni ve gelecek hedefler.

12. Hızlı Başvuru Kartı (Cheat Sheet)

Tek bakışta sözdizimi, anahtar sözcükler ve kısayollar tablosu.

1. Giriş ve Dilin Temel Felsefesi

MRT, standart C (C11/C17) diliyle sıfırdan geliştirilmiş modern, dinamik tipli ve temiz mimarili bir programlama dilidir. MRT projesinin temel amacı; harici kod oluşturuculara (Flex, Bison, ANTLR vb.) ihtiyaç duymadan, baştan sona saf C standart kütüphanesi kullanılarak endüstri standartlarında bir dil çekirdeğinin nasıl inşa edilebileceğini somutlaştırmaktır.

Temel Tasarım İlkeleri:

- Sıfır Dış Bağımlılık (Standard C Only):** Dil yorumlayıcısı yalnızca ANSI/ISO C11 standart kütüphanesini kullanır.
- El Yapımı Ön Yüz (Handwritten Frontend):** Lexer (sözcüksel analiz) ve Recursive Descent Parser tamamen özel olarak kodlanmıştır.
- Ayrık Katman Mimarisi:** Dil ön yüzü (AST) ile çalışma zamanı (Interpreter) birbirinden kesin sınırlarla ayrılmıştır.
- Mutlak Bellek Güvenliği:** Referans sayımlı bellek yönetimi uygulanmış olup, AddressSanitizer (ASan) ve LeakSanitizer (LSan) ile **0 bayt bellek sızıntısı** doğrulanmıştır.
- Özgün Sözdizimi Kimliği:** MRT 0.2.0 sürümüyle birlikte jenerik kalıplar bırakılmış; var, task, give, say, when, repeat gibi özgün anahtar sözcükler benimsenmiştir.

MRT 0.2.0 Dönüm Noktası: MRT, 0.1 sürümündeki genel JavaScript/Python benzeri sözdizimini geride bırakarak kendi dil kimliğini kazanmıştır. Bu dokümanda yer alan tüm örnekler ve açıklamalar güncel MRT 0.2.0 sözdizimini esas almaktadır.

2. Kurulum ve Hızlı Başlangıç

Gereksinimler:

- C Derleyicisi: **GCC (11+)** veya **Clang (13+)**
- Derleme Aracı: **GNU Make**
- İşletim Sistemi: Modern Linux dağıtımları (Ubuntu, Debian, Fedora, Arch, vb.)

Derleme Komutları:

```
# 1. Depoyu klonlayın
git clone https://github.com/M0zcelik14/MRT.git
cd MRT

# 2. MRT yorumlayıcısını derleyin
make

# 3. Test paketini çalıştırın (AddressSanitizer aktif)
make test

# 4. MRT Studio IDE'sini derleyin
make studio

# 5. Sisteme kullanıcı düzeyinde (~/.local/bin) kurun
make install PREFIX=~/.local
```

İlk Programınız: Merhaba Dünya!

MRT kaynak dosyaları .mrt uzantısını taşır. Favori metin editörünüzle merhaba.mrt adında bir dosya oluşturun:

```
// merhaba.mrt
say "Merhaba Dünya! MRT 0.2.0 çalışıyor."
```

Programı terminalden çalıştırmak için:

```
mrt merhaba.mrt
```

İnteraktif REPL Modu:

MRT, dosya adı vermeksizin çalıştırıldığında anlık deneme yapabileceğiniz interaktif REPL moduna geçer:

```
$ mrt
MRT 0.2.0 Interactive REPL
Type 'exit' to quit.

mrt> var x = 15 * 3
mrt> say x
45
mrt> say toText(x) + " elma"
45 elma
mrt> exit
```

Komut Satırı Bayrakları:

- mrt <dosya.mrt>: Kaynak dosyayı yorumlar ve çalıştırır.
- mrt --tokens <dosya.mrt>: Sözcüksel analiz sonucundaki token akışını döker.

- `mrt --ast <dosya.mrt>`: Oluşturulan Soyut Sözdizim Ağacını (AST) hiyerarşik olarak yazdırır.
- `mrt --version`: MRT sürüm numarasını görüntüler.
- `mrt --help`: Kullanım yardımını gösterir.

3. MRT 0.2.0 Sözdizimi ve Veri Tipleri

Yorum Satırları:

MRT'de tek satırlık ve çok satırlı blok yorumlar desteklenir:

```
// Tek satırlık açıklama
var pi = 3.14

/*
    Bu bir blok yorumdur.
    Birden fazla satıra yayılabilir.
*/
var yarıcap = 5
```

Değişkenler (Variables):

Değişken tanımlamak için var anahtar sözcüğü kullanılır. MRT'de noktalı virgül (;) isteğe bağlıdır; yeni satırlar deyim sonlandırıcı kabul edilir:

```
var isim = "Murat"
var yas = 22
yas = yas + 1 // Değer yeniden atama
```

Veri Tipleri (Data Types):

Tip	Örnek Değerler	Açıklama
integer	10, -5, 0, 1000000	64-bit işaretli tamsayı (int64_t).
float	3.14, -0.05, 2.0	64-bit çift duyarlıklı kayan noktalı sayı (double).
string	"Merhaba\nDünya"	Dinamik boyutlu UTF-8 metin dizgisi.
boolean	yes, no	Mantıksal doğruluk (yes) veya yanlışlık (no).
none	none	Boşluk/yokluk değeri (null/nil dengi).
function	task kare(x) { ... }	Birinci sınıf fonksiyon/görev nesnesi.

Dizgelerde Kaçış Karakterleri (Escape Sequences):

Dizgeler çift tırnak ("...") ile çevrelenir ve şu kaçış karakterlerini destekler:

- \n : Yeni satır (newline)
- \t : Sekme boşluğu (tab)
- \" : Çift tırnak karakteri
- \\ : Ters eğik çizgi (backslash)

4. Operatörler ve İfadeler

MRT, zengin aritmetik, karşılaştırma ve mantıksal operatörlere sahiptir. Operatörler standart matematiksel öncelik kurallarına (precedence climbing) göre kesin bir hiyerarşide değerlendirilir:

Öncelik	Operatör	Tür	Yön
1 (En Düşük)	=	Atama	Sağdan Sola
2	or	Mantıksal VEYA	Soldan Sağa
3	and	Mantıksal VE	Soldan Sağa
4	==, !=	Eşitlik / Eşitsizlik	Soldan Sağa
5	<, <=, >, >=	Büylük / Küçüklük	Soldan Sağa
6	+, -	Toplama / Çıkarma	Soldan Sağa
7	*, /, %	Çarpma / Bölme / Mod	Soldan Sağa
8	-, not	Tekil Eksi / Değil	Önek (Prefix)
9	()	Fonksiyon Çağrısı	Sonek (Postfix)
10 (En Yüksek)	Değer, (ifade)	Temel / Parantez	—

5. Konsol Çıktısı: 'say' Deyimi

MRT 0.2.0'da konsola metin ve değer yazdırmak için say deyimi kullanılır. say bir fonksiyon çağırısı (print(...)) değil, dilin çekirdek sözdizimine yerleşik **birinci sınıf bir deyimdir (statement)**.

Özellikleri:

- Parantez gerektirmez: say <ifade> biçiminde çalışır.
- İfadeyi otomatik olarak hesaplar ve ekrana yazdırır.
- Çıktının sonuna daima otomatik yeni satır (\n) ekler.
- Dizge olmayan değerleri de doğrudan yazdırabilir.

```
say "Merhaba!"
say 42
say 3.14 * 2
say yes
say none

// Metin birleştirme ile yazdırma:
var kullanıcı = "Murat"
say "Hoş geldin, " + kullanıcı + "!"
```

6. Kontrol Akışı (when, otherwise, repeat)

Koşullu İfadeler (when / otherwise):

MRT'de geleneksel if / else yerine when / otherwise blokları kullanılır. Bu yapı koda hem okunabilirlik hem de doğal bir dil akışı kazandırır:

```
var notOrtalamasi = 85

when notOrtalamasi >= 90 {
  say "Harf Notu: AA"
} otherwise when notOrtalamasi >= 80 {
  say "Harf Notu: BA"
} otherwise when notOrtalamasi >= 70 {
  say "Harf Notu: BB"
} otherwise {
  say "Kaldınız."
}
```

Döngüler (repeat) ve Akış Kontrolü (break, continue):

Yinelenen işlemler için repeat (durum) döngüsü kullanılır. Döngüyü anında sonlandırmak için break, o adımı atlayıp bir sonraki yinelemeye geçmek için continue deyimleri kullanılır:

```
var sayac = 0
var toplam = 0

repeat sayac < 10 {
    sayac = sayac + 1

    // 5 sayısını toplama dahil etme, atla:
    when sayac == 5 {
        continue
    }

    // 8'e ulaşıldığında döngüyü tamamen sonlandır:
    when sayac == 8 {
        break
    }

    toplam = toplam + sayac
    say "Sayac: " + toText(sayac) + ", Ara Toplam: " + toText(toplam)
}

say "Nihai Toplam: " + toText(toplam)
```

7. Görevler (Tasks / Fonksiyonlar)

MRT'de fonksiyonlar task anahtar sözcüğü ile tanımlanır. Bir görevden değer döndürmek için give deyimi kullanılır. Eğer bir görevde give çağrılmazsa, görev varsayılan olarak none değeri üretir.

```
// Temel Görev Tanımı
task topla(a, b) {
  give a + b
}

var sonuc = topla(15, 25)
say "Sonuç: " + toText(sonuc) // 40
```

Sözcüksel Kapsam ve Değişken Gölgeleme (Scoping & Shadowing):

Her görev ve kod bloğu ({ ... }) yeni bir sözcüksel kapsam (lexical scope) oluşturur. İç blokta tanımlanan bir değişken dış bloktaki değişkeni geçici olarak gölgeler, dış değişkenin değerini bozmaz:

```
var x = 100

task ornek() {
  var x = 50 // Dıştaki x'i gölgeler
  say "Görev içi x: " + toText(x) // 50
}

ornek()
say "Görev dışı x: " + toText(x) // 100 (değişmedi)
```

Özyineleme (Recursion):

MRT, özyinelemeli fonksiyon çağrılarını eksiksiz destekler:

```
// Faktöriyel Hesabı
task faktoriyel(n) {
  when n <= 1 {
    give 1
  }
  give n * faktoriyel(n - 1)
}

say "5! = " + toText(faktoriyel(5))  // 120

// Fibonacci Hesabı
task fib(n) {
  when n <= 1 {
    give n
  }
  give fib(n - 1) + fib(n - 2)
}

say "fib(10) = " + toText(fib(10))  // 55
```

8. Yerleşik Standart Kütüphane

MRT çekirdeğinde kullanıma hazır temel yerleşik işlevler yer alır:

Fonksiyon	Dönüş Tipi	Açıklama ve Kullanım Örneği
typeof(değer)	string	Verilen değerin tip adını metin olarak döner: "integer", "float", "string", "boolean", "none", "function".
length(metin)	integer	Verilen dizgenin karakter sayısını döner: length("MRT") -> 3.
toText(değer)	string	Herhangi bir değeri metne dönüştürür. Sayılarla dizgeleri birleştirirken kullanılır.
clock()	float	Programın başladığı andan itibaren geçen CPU süresini saniye cinsinden döner.

9. Hata Yönetimi ve Tanılama

MRT, modern derleyicilerdekine (GCC, Clang, Rust) benzer nitelikte gelişmiş hata teşhis mekanizmasına sahiptir. Bir hata meydana geldiğinde yalnızca mesaj basılmaz; kaynak dosyanın ilgili satırı, sütunu ve hatanın tam konumu imleç (^) işaretiyle görsel olarak gösterilir.

Hata Kategorileri:

- **SyntaxError:** Sözdizimi kuralı ihlalleri (eksik parantez, geçersiz token, vb.).
- **NameError:** Tanımlanmamış bir değişkene veya göreve erişim denemesi.
- **TypeError:** Uyumsuz tipler arası işlem (örneğin bir tamsayı ile mantıksal değeri toplama).
- **RuntimeError:** Çalışma zamanında sıfıra bölme gibi aritmetik veya mantıksal hatalar.

Örnek Görsel Hata Çıktısı:

```
ornek.mrt:4:12
```

```
    say 100 / 0  
              ^
```

```
RuntimeError: division by zero
```

10. MRT Studio IDE Kılavuzu

MRT Studio, MRT geliştiricileri için sıfırdan **GTK4** ve **GtkSourceView 5** kullanılarak C diliyle yazılmış yerel Linux geliştirme ortamıdır. Web teknolojileri veya ağır runtime'lar barındırmaz; açılış anlıktır ve bellek tüketimi düşüktür.

Öne Çıkan IDE Yetenekleri:

- **Çift Dilli Arayüz (TR / EN):** Sistem diline otomatik uyum sağlar. Ayarlar penceresinden anında Türkçe veya İngilizce seçilebilir. Yedek sözlük mimarisi sayesinde sistemde tr_TR yereli olmasa da çalışır.
- **Komut Paleti (Ctrl+Shift+P):** Tüm IDE komutlarını klavyeden arayıp tek tuşla tetikleyebileceğiniz modern komut çubuğu.
- **Hızlı Dosya Aç (Ctrl+P):** Projedeki tüm kaynak dosyaları anında filtreleyip doğrudan düzenleme sekmesine getiren araç.
- **Dosyalarda Bul (Ctrl+Shift+F):** Proje genelindeki tüm .mrt dosyalarında metin araması yapar; eşleşen satıra ve sütuna tıklandığında imleci doğrudan hata/kod satırına götürür.
- **Satıra Git (Ctrl+G):** Büyük dosyalarda istenen satır numarasına doğrudan atlama kutusu.
- **Sembol Ağacı (Outline):** Sol kenar çubuğunda açılan dosyada tanımlı tüm 'task' ve 'var' sembollerini gerçek zamanlı listeler; tıklandığında tanımlandığı satıra odaklanır.

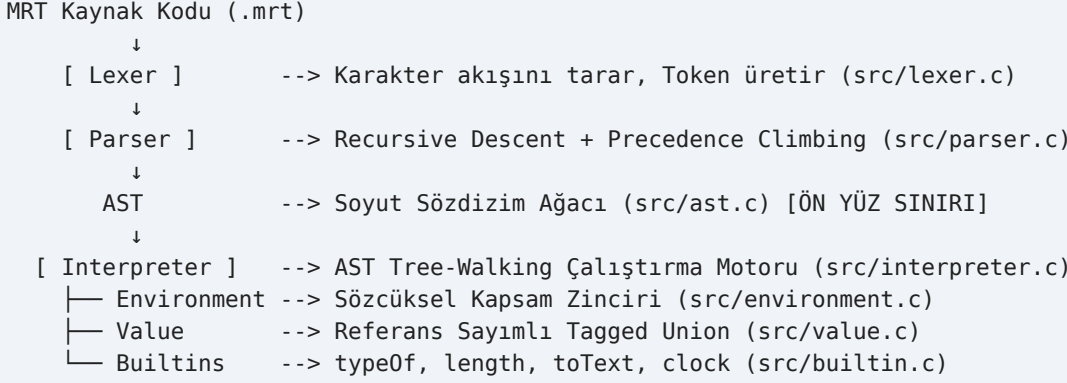
- **Sorunlar & Çıktı Paneli:** MRT programlarının stdout/stderr akışını asenkron gösterir. Hata satırları parse edilir ve Sorunlar sekmesinde tıklanabilir hata listesi sunulur.
- **Otomatik Yedekleme & Kurtarma:** 15 saniyede bir değiştirilen dosyaları ~/.local/state/mrt-studio/recovery/ dizininde güvene alır. Olası çökme veya elektrik kesintisinde açılışta dosyaları geri yükler.
- **Yeni Proje Sihirbazı:** Konsol Uygulaması (Console), Boş Proje (Empty) ve Kütüphane (Library) şablonları ile saniyeler içinde yeni proje iskeleti kurar.

Klavye Kısayolları Tablosu:

Kısayol	Eylem	Kısayol	Eylem
Ctrl+N	Yeni Dosya	F5	Programı Çalıştır (Run)
Ctrl+O	Dosya Aç	Ctrl+Shift+P	Komut Paleti
Ctrl+S	Kaydet	Ctrl+P	Hızlı Dosya Aç
Ctrl+Shift+S	Farklı Kaydet	Ctrl+Shift+F	Dosyalarda Bul
Ctrl+Alt+S	Tümünü Kaydet	Ctrl+G	Satıra Git
Ctrl+F	Bul	Ctrl+B	Kenar Çubuğunu Gizle/Aç
Ctrl+H	Bul & Değiştir	Ctrl+J	Çıktı Panelini Gizle/Aç
Ctrl+Z	Geri Al (Undo)	Ctrl+Q	MRT Studio'dan Çıkış

11. Dil Mimarisi ve Gelecek Yol Haritası

MRT dil motoru, derleyici tasarımının temel ilkelerine tam sadakatle inşa edilmiştir. Mimari, temiz bir modüler ayırım ve genişletilebilirlik üzerine kuruludur:



Gelecek Yol Haritası (Roadmap):

- **MRT 0.3.0 — Bytecode VM:** AST'den doğrudan bytecode derlemesi ve yığın tabanlı (stack-based) hızlı bir sanal makine geliştirilmesi.
- **MRT 0.4.0 — C Transpiler:** MRT kodlarını optimize edilmiş C koduna dönüştürerek GCC/Clang ile doğrudan yerel ikili (native binary) üretme kabiliyeti.
- **MRT Paket Yöneticisi:** Kütüphaneleri ve modülleri kolayca yükleyip yönetecek CLI aracı.
- **Standart Kütüphane Genişlemesi:** Dosya I/O, matematik ve ağ socket desteği.

12. Hızlı Başvuru Kartı (Cheat Sheet)

```
// MRT 0.2.0 Tam Özet Kod Bloğu

// 1. Değişkenler ve Tipler
var tamsayi = 42
var ondalik = 3.14
var metin = "MRT Dili"
var aktif = yes
var pasif = no
var bos = none

// 2. Konsola Yazdırma
say "Sayı: " + toText(tamsayi)

// 3. Koşul Yapısı
when tamsayi > 50 {
    say "Büyük"
} otherwise when tamsayi == 42 {
    say "Doğru Cevap!"
} otherwise {
    say "Küçük"
}

// 4. Döngü ve Akış Kontrolü
var i = 0
repeat i < 5 {
    i = i + 1
    when i == 2 { continue }
    when i == 4 { break }
    say "i = " + toText(i)
}

// 5. Görev Tanımı ve Çağrısı
task topla(a, b) {
    give a + b
}

say "10 + 20 = " + toText(topla(10, 20))
```